

Implementación Cloud: Colegio BO

Presentado por:

Alejandro Francis
Davier Ramos
Dilan Micolta
Vicente Manriquez

Profesor:

Ricardo Aravena

índice

Entra ID.....	3
Base de datos de AWS.....	7
Test en postman antes de integrar EC2.....	13
Integrar los EC2.....	18
Subir los microservicios al EC2.....	21
API GATEWAY.....	25
JWT AUTHORIZER.....	28
Frontend MSAL.....	30

Entra ID

1. Crear el tenant

2. Registrar aplicacion para el backend

Registro de la app backend: se registra "Pedidos360-API" en Azure como aplicación de inquilino único, sin URI de redirección.

Microsoft Azure

Buscar recursos, servicios y documentos (G+/I)

Copilot

Inicio

Registrar una aplicación

* Nombre

Nombre para mostrar accesible por los usuarios de esta aplicación. Se puede cambiar posteriormente.

Pedidos360-API

Típos de cuenta compatibles

Elija los tipos de cuenta que podrán usar esta aplicación o acceder a esta API

Solo inquilino único: VicenteManriquez [Ayúdeme a elegir](#)

URI de redirección (opcional)

Devolveremos la respuesta de autenticación a esta dirección URI después de autenticar correctamente al usuario. Este dato es opcional y se puede cambiar más tarde, pero se necesita un valor para la mayoría de los escenarios de autenticación.

Seleccionar una plataforma

Registre una aplicación en la que esté trabajando aquí. Integre aplicaciones de la galería y otras aplicaciones de fuera de la organización agregándolas desde [Aplicaciones empresariales](#).

Al continuar, acepta las directivas de la plataforma Microsoft.

Registrar

3. Agregar un URI de id de app

Agregar URI de id. de aplicación: se hace clic en el enlace que permite definir el identificador de la API.

URI de id. de aplicación : [Agregar un URI de id. de aplicación](#)

Agregar un ámbito

Necesitará establecer un URI de id. de aplicación antes de agregar un permiso. Hemos seleccionado uno, pero puede cambiarlo.

URI de id. de aplicación * ⓘ

Guardar y continuarCancelar

4. Configurar scope del URI

Agregar un ámbito

Nombre de ámbito * ⓘ

api://8be95624-6fa3-4b6f-b4c2-76578057c9a1/access_as_user

¿Quién puede dar el consentimiento? ⓘ

Administradores y usuarios Solo administradores

Nombre para mostrar del consentimiento del administrador * ⓘ

Descripción del consentimiento del administrador * ⓘ

Nombre para mostrar del consentimiento del usuario ⓘ

y 4. **Cuadro "Agregar un ámbito"**: Azure propone el URI `api://<id-de-cliente>` y se confirma con "Guardar y continuar" (es la misma captura usada dos veces).

Creación del scope `access_as_user`: consentimiento para administradores y usuarios, con el nombre "Acceder a Pedidos360 API" y estado habilitado.

5. Crear roles

Roles de aplicación: se crean tres roles (Admin, Vendedor y Cliente), asignables a usuarios o grupos.

Roles de aplicación

Los roles de aplicación son roles personalizados que sirven para asignar permisos a usuarios o aplicaciones. La aplicación define y publica los roles de aplicación y los interpreta como permisos durante la autorización.

¿Cómo se asignan roles de aplicación?

Nombre para mostrar	Descripción	Tipos de miembro perm...	Valor	Id.	Estado
Admin	es el admin	Usuarios o grupos	Admin	fd0d1cdd-9d82-4312-...	Habilitado
Vendedor	crear y gestionar pedidos	Usuarios o grupos	Vendedor	a439496b-7207-4de6-...	Habilitado
Cliente	consultar y realizar pedidos	Usuarios o grupos	Cliente	44804a30-a9cb-4396-...	Habilitado

6. Crear app del frontend (mismo url que vite)

Registro de la app frontend: "Pedidos360-Frontend" como aplicación de página única, con redirección a <http://localhost:5173/> (el puerto de Vite).

Registrar una aplicación ...

* Nombre

Nombre para mostrar accesible por los usuarios de esta aplicación. Se puede cambiar posteriormente.

Pedidos360-Frontend ✓

Tipos de cuenta compatibles

Elija los tipos de cuenta que podrán usar esta aplicación o acceder a esta API

Solo inquilino único: VicenteManriquez

[Ayúdeme a elegir](#)

URI de redirección (opcional)

Devolveremos la respuesta de autenticación a esta dirección URI después de autenticar correctamente al usuario. Este dato es opcional y se puede cambiar más tarde, pero se necesita un valor para la mayoría de los escenarios de autenticación.

Aplicación de página única (...)

<http://localhost:5173/> ✓

Registre una aplicación en la que esté trabajando aquí. Integre aplicaciones de la galería y otras aplicaciones de fuera de la organización agregándolas desde [Aplicaciones empresariales](#).

7. Asignar Pedidos API a Pedidos Frontend

Permisos de API: el frontend solicita el permiso delegado `access_as_user` de Pedidos360-API, sin consentimiento de administrador.

Solicitud de permisos de API



[< Todas las API](#)

 Pedidos360-API
api://8be95624-6fa3-4b6f-b4c2-76578057c9a1

¿Qué tipo de permiso necesita la aplicación web?

Permisos delegados

Su aplicación necesita acceder a la API como el usuario que haya iniciado la sesión.

Permisos de la aplicación

Su aplicación se ejecuta como servicio en segundo plano o demonio sin un usuario que haya iniciado la sesión.

Seleccionar permisos

[expandir todo](#)

 Empiece a escribir un permiso para filtrar los resultados.

 La columna "Se necesita el consentimiento del administrador" muestra el valor predeterminado de una organización. No obstante, el consentimiento del usuario puede personalizarse por permiso, por usuario o por aplicación. Es posible que esta columna no refleje el valor de su organización o el valor de las organizaciones en las que se usará esta aplicación. [Más información](#)

Permiso	Se necesita el consentimiento del a...
▼ Permisos (1)	
☒ access_as_user ⓘ Acceder a Pedidos360 API	No

Agregar permisos

Descartar

Base de datos de AWS

1. Buscamos rds en aws console
2. Entramos a aurora rds y seleccionamos crear y elegimos los parametros

Motor y método: se elige PostgreSQL con "Configuración completa".

Crear base de datos [Información](#)

Opciones del motor

Tipo de motor [Información](#)

☐ Aurora (MySQL Compatible)	☐ Aurora (PostgreSQL Compatible)	☐ MySQL	☒ PostgreSQL
☐ MariaDB	☐ Oracle	☐ Microsoft SQL Server	☐ IBM Db2

Elegir un método de creación de base de datos

☒ **Configuración completa**
Puede definir todas las opciones de configuración, incluidas las de disponibilidad, seguridad, copias de seguridad y mantenimiento.

☐ Creación sencilla
Utilice las configuraciones recomendadas. Algunas opciones de configuración se pueden cambiar después de crear la base de datos.

Plantilla y disponibilidad: plantilla "Capa gratuita", con una sola instancia en una zona de disponibilidad, sin redundancia.

Plantillas

Elija una plantilla de ejemplo para adaptarla a su caso de uso.

☐ Producción
Utilice los valores predeterminados para disfrutar de una alta disponibilidad y de un rendimiento rápido y constante.

☐ Desarrollo y pruebas
Esta instancia se ha diseñado para su uso en desarrollo, fuera de un entorno de producción.

☒ **Capa gratuita**
Utilice el nivel gratuito de RDS para desarrollar nuevas aplicaciones, probar aplicaciones existentes o adquirir experiencia práctica con Amazon RDS. [Información](#)

Disponibilidad y durabilidad

Opciones de implementación [Información](#)

Elija la opción de implementación que proporcione la disponibilidad y durabilidad necesarias en función del caso de uso. AWS se compromete a un determinado nivel de tiempo de actividad según la opción de implementación que elija. Obtenga más información en el [Acuerdo de nivel de servicios \(SLA\) de Amazon RDS](#).

☐ Implementación de clúster de base de datos multi-AZ (3 instancias)
Crea una instancia de base de datos principal con dos en espera legibles en zonas de disponibilidad separadas. Esta configuración proporciona:
• Tiempo de actividad del 99,95 %
• Redundancia entre zonas de disponibilidad
• Mayor capacidad de lectura
• Menor latencia de escritura

☐ Implementación de instancias de base de datos multi-AZ (2 instancias)
Crea una instancia de base de datos principal con una instancia en espera no legible en una zona de disponibilidad independiente. Esta configuración proporciona:
• Tiempo de actividad del 99,95 %
• Redundancia entre zonas de disponibilidad

☒ **Implementación de una instancia de base de datos de zona de disponibilidad única (1 instancia)**
Crea una única instancia de base de datos sin instancias en espera. Esta configuración proporciona:
• Tiempo de actividad del 99,5 %
• Sin redundancia de datos

Punto de conexión de escritura/lectura

Puntos de conexión del lector

En espera legible + SSD

En espera

Instancia principal + SSD

Instancia principal

Instancia principal

Credenciales: identificador `database-1`, usuario maestro `postgres` y contraseña autoadministrada. Azure marca la contraseña como "Very weak".

Identificador de instancias de bases de datos [Información](#)
 Escriba un nombre para la instancia de base de datos. El nombre debe ser único en relación con todas las instancias de base de datos pertenecientes a su cuenta de AWS en la región de AWS actual.

database-1

El identificador de la instancia de base de datos no distingue entre mayúsculas y minúsculas, pero se almacena con todas las letras en minúsculas (como en "mydbinstance"). Restricciones: de 1 a 63 caracteres alfanuméricos o guiones. El primer carácter debe ser una letra. No puede contener dos guiones consecutivos. No puede terminar con un guion.

Configuración de credenciales

Nombre de usuario maestro [Información](#)
 Escriba un ID de inicio de sesión para el usuario maestro de la instancia de base de datos.

postgres

1 a 16 caracteres alfanuméricos. El primer carácter debe ser una letra.

Administración de credenciales
 Puede usar AWS Secrets Manager o administrar sus credenciales de usuario maestro.

☐ Administrado en AWS Secrets Manager - *más seguro*
 RDS genera una contraseña y la administra durante todo su ciclo de vida mediante AWS Secrets Manager.

☒ Autoadministrado
 Cree su propia contraseña o pida a RDS que cree una contraseña para que pueda administrarla.

☐ Generar contraseña automáticamente
 Amazon RDS puede generar una contraseña en su nombre, o bien puede especificar su propia contraseña.

Contraseña maestra [Información](#)

Password strength Very weak

Restricciones mínimas: al menos 8 caracteres ASCII imprimibles. No puede contener ninguno de los siguientes símbolos: / ' " @

Confirmar la contraseña maestra [Información](#)

12345678

Instancia: clase ampliable **db.t3.micro**

Configuración de la instancia
 Las opciones de configuración de la instancia de base de datos que aparecen a continuación están limitadas a las que admite el motor que ha seleccionado anteriormente.

Clase de instancia de base de datos [Información](#)

▼ Ocultar filtros

☒ Incluir clases de generación anterior

☐ Clases estándar (incluye clases m)

☐ Clases optimizadas para memoria (incluye clases r y x)

☒ Clases ampliables (incluye clases t)

Tipo de instancia

db.t3.micro
 2 vCPUs 1 GiB RAM Ancho de banda de EBS: hasta 2085 Mbps Red: hasta 5 Gbps

Almacenamiento: SSD de uso general (gp2) de 20 GiB.

Almacenamiento

Tipo de almacenamiento [Información](#)
 Los volúmenes de almacenamiento SSD de IOPS aprovisionadas (io2) ya están disponibles.

SSD de uso general (gp2)
 Rendimiento de referencia determinado por el tamaño del volumen

Almacenamiento asignado [Información](#)

20 GiB

El valor de almacenamiento asignado debe ser de 20 GiB a 6144 GiB

► Configuración de almacenamiento adicional

Red: VPC por defecto, acceso público "Sí" y un grupo de seguridad nuevo llamado **Pedidos360-db-sg**.

☒ No se conecte a un recurso informático EC2
No configure una conexión a un recurso informático para esta base de datos. Puede configurar manualmente una conexión a un recurso informático más adelante.

☐ Conectarse a un recurso informático de EC2
Configure una conexión a un recurso informático EC2 para esta base de datos.

Listado de RDS: la base **pedidos360** aparece "Disponible" con motor PostgreSQL.

Bases de datos (1)				
Filtrar por bases de datos				
Identificador de base de datos	Estado	Rol	Motor	
pedidos360	Disponible	Instancia	PostgreSQL	

- Entrar a la base de datos y copiar los puertos y url
export RDSHOST="pedidos360.cdkqqe6kw6u1.us-east-1.rds.amazonaws.com"
psql "host=\$RDSHOST port=5432 dbname=postgres user=postgres sslmode=verify-full sslrootcert=./global-bundle.pem"

Pasos de conexión de AWS: el comando **curl** para bajar **global-bundle.pem**, y el **export RDSHOST** con el **psql (sslmode=verify-full)**.

Lenguaje de programación
Tipo de punto de conexión

PSQL (macOS)
Punto de conexión de la instancia

Pasos de conexión
Siga los pasos que se indican a continuación para pegar el código de cada paso en la herramienta y ejecutar los comandos. Los fragmentos reflejan de forma dinámica la configuración de autenticación.

```
1 curl -o global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
```

```
1 export RDSHOST="pedidos360.cdkqqe6kw6u1.us-east-1.rds.amazonaws.com"
```

```
2 psql "host=$RDSHOST port=5432 dbname=postgres user=postgres sslmode=verify-full sslrootcert=./global-bundle.pem"
```

- Probar conexión con la base de aws desde nuestra maquina
Registramos el servidor con los parametros adecuados

pgAdmin, pestaña General: se registra el servidor "Pedidos360-RDS".

The screenshot shows the 'Register - Server' dialog box in pgAdmin, with the 'General' tab selected. The dialog has a dark theme and a close button (X) in the top right corner. The tabs are: General, Connection, Parameters, SSH Tunnel, Advanced, Post Connection SQL, and Tags. The 'Name' field contains 'Pedidos360-RDS'. The 'Server group' dropdown is set to 'Servers'. There are 'X' buttons for 'Background' and 'Foreground'. The 'Connect now?' toggle is turned on. The 'Comments' field is empty. At the bottom, there is an error message: 'Either Host name or Service must be specified.' with a red 'X' icon. The bottom bar contains an info icon, a help icon, and buttons for 'Close', 'Reset', and 'Save'.

Register - Server

General Connection Parameters SSH Tunnel Advanced Post Connection SQL Tags

Name Pedidos360-RDS

Server group Servers

Background X

Foreground X

Connect now? ☒

Comments

ⓘ ? X Close ↺ Reset 💾 Save

❗ Either Host name or Service must be specified.

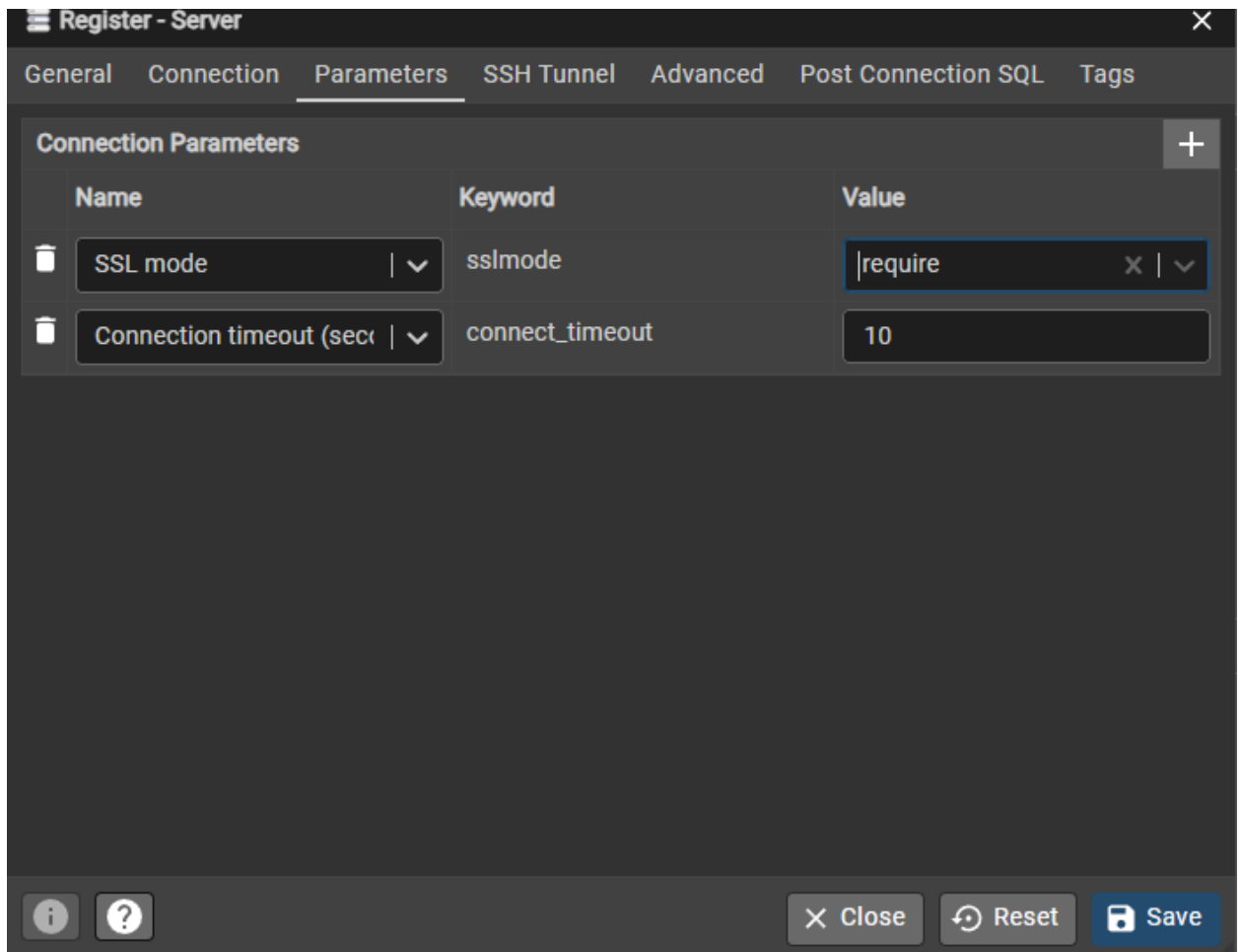
pgAdmin, pestaña Connection: se completan host de RDS, puerto 5432, base **postgres**, usuario y contraseña.

The screenshot shows the 'Register - Server' dialog box in pgAdmin, with the 'Connection' tab selected. The dialog has a dark theme and a close button (X) in the top right corner. The tabs at the top are: General, Connection (selected), Parameters, SSH Tunnel, Advanced, Post Connection SQL, and Tags. The fields and their values are:

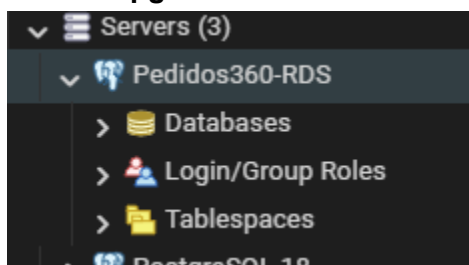
- Host name/address: pedidos360.cdkqqe6kw6u1.us-east-1.rds.amazonaws.com
- Port: 5432
- Maintenance database: postgres
- Username: postgres
- Kerberos authentication?: ☐
- Password: (masked)
- Save password?: ☒
- Role: (empty field)
- Service: (empty field)

Below the password field, there is a note: "In edit mode the password field is enabled only if Save Password is set to true." At the bottom of the dialog, there are three buttons: 'Close' (with an X icon), 'Reset' (with a circular arrow icon), and 'Save' (with a floppy disk icon).

pgAdmin, pestaña Parameters: **sslmode=require** y timeout de conexión de 10 s.



Árbol de pgAdmin: el servidor Pedidos360-RDS ya aparece registrado.



5. Creamos las 6 bases de datos para los 6 microservicios

Bases creadas: academico_db, asistencia_db, matricula_db, mensajeria_db, reportes_db, usuarios_db, además de las por defecto.



Test en postman antes de integrar EC2

1. Configuramos el proyecto para añadir la base de datos de aws y el login de azure

Archivo de propiedades de Spring: datasource hacia RDS con la contraseña por variable de entorno `DB_PASSWORD`, `ddl-auto=update`, validación JWT contra Azure AD y RabbitMQ local.

```
# --- Base de datos ---
# Conexión a RDS PostgreSQL (AWS)
spring.datasource.url=jdbc:postgresql://pedidos360.cdkqqe6kw6u1.us-east-1.rds.amazonaws.com:5432/usuarios_db?sslmode=require
spring.datasource.username=postgres
# La contraseña se inyecta desde la variable de entorno DB_PASSWORD (nunca hardcodeada aquí)
spring.datasource.password=${DB_PASSWORD}

# Driver y Dialecto
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect

# 'update' + crea/modifica tablas sin borrar datos existentes
spring.jpa.hibernate.ddl-auto=update

# Mostrar las consultas SQL en la consola para debuggear
spring.jpa.show-sql=true

# Azure AD - valida la firma del token sin importar la versión (v1.0 o v2.0)
spring.security.oauth2.resourceserver.jwt.jwk-set-uri=https://login.microsoftonline.com/4646e528-a2b6-4aac-b415-5b0ddd4897d8/discovery/v2.0

# --- RabbitMQ ---
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
```

2. Agregar a postman en secrets

Secreto de cliente en Azure: se crea el secreto "postman" con su fecha de expiración.

+ Nuevo secreto de cliente			
Descripción	Expira	Valor ⓘ	Id. de secreto
postman	12/3/2027	f2p8Q~w1H_2gLdTR~CrP8nRghXYNJnA...	406b1193-c4ec-4de3-98e2-2aa39f73c5a0 ⓘ

Nuevo token en Postman: grant type Authorization Code, con las URLs de Microsoft, el client ID y secret, y el scope `api://...`

Configure New Token

Token Name

Azure Token

Grant type

Authorization Code

Callback URL ⓘ

https://oauth.pstmn.io/v1/callback

☐ Authorize using browser

Auth URL ⓘ

https://login.microsoftonline.com/4646e52...

Access Token URL ⓘ

https://login.microsoftonline.com/4646e52...

Client ID ⓘ

..... ⓘ ⚠

Client Secret ⓘ

..... ⓘ ⚠

Scope ⓘ

api://8be95624-6fa3-4b6f-b4c2-7657805:...

State ⓘ

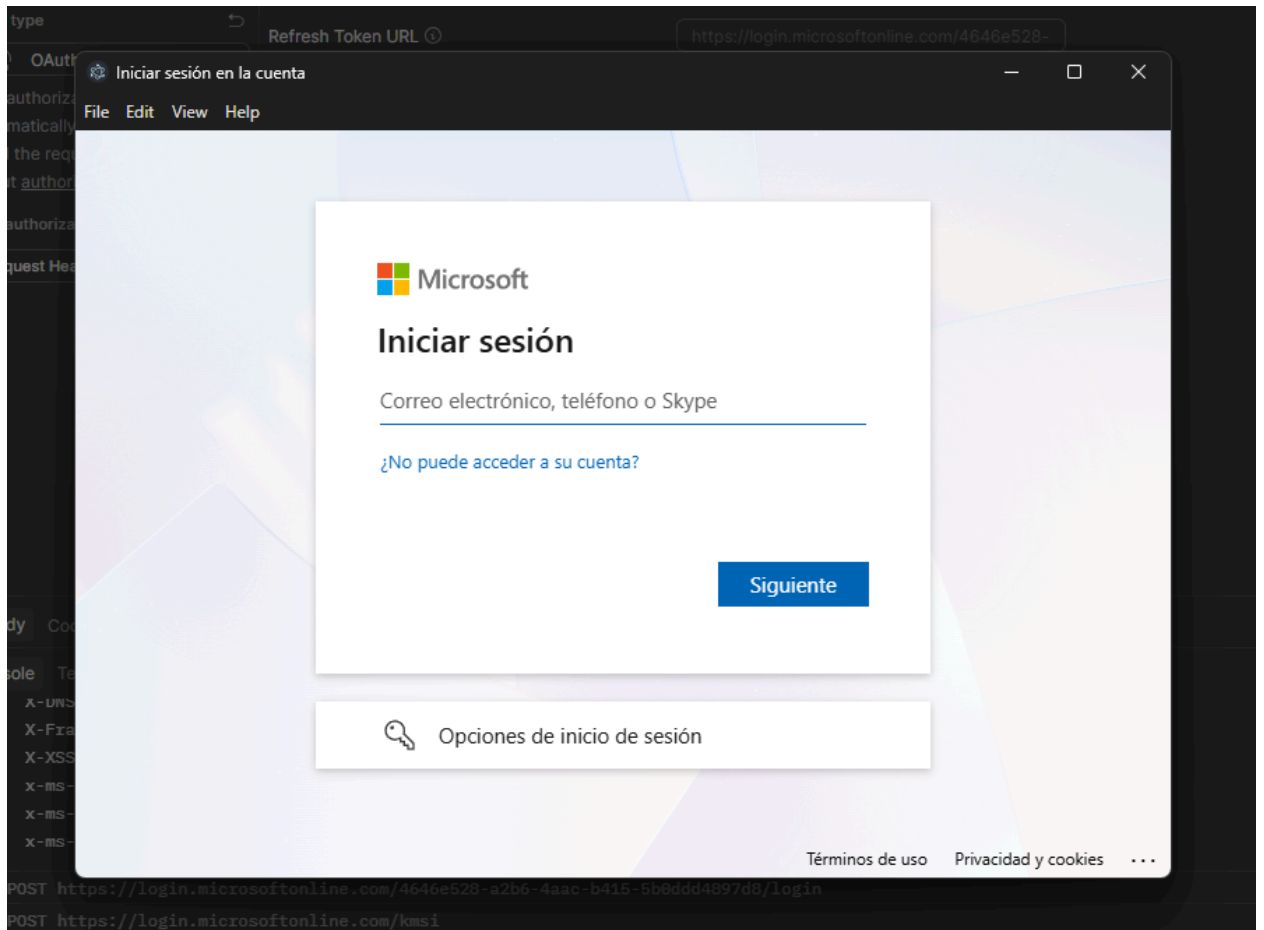
State

Client Authentication ⓘ

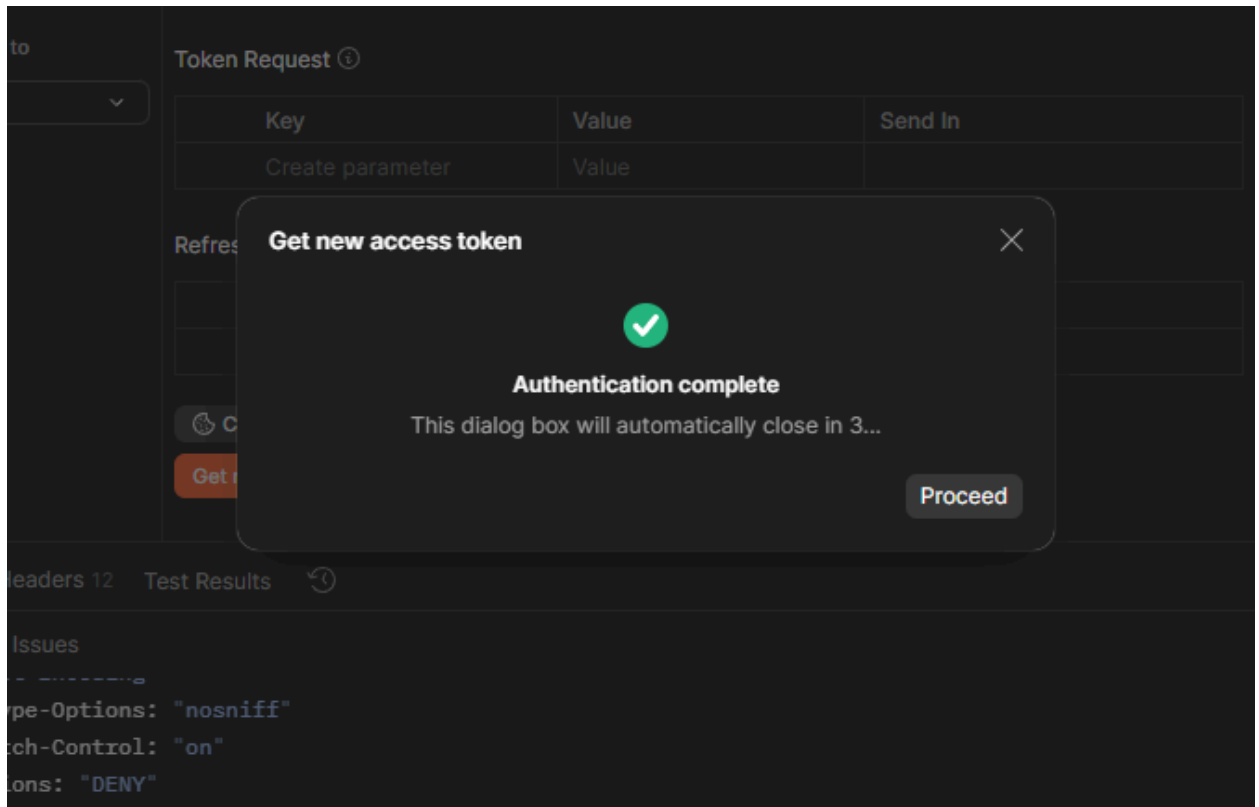
Send client credentials in body

Advanced

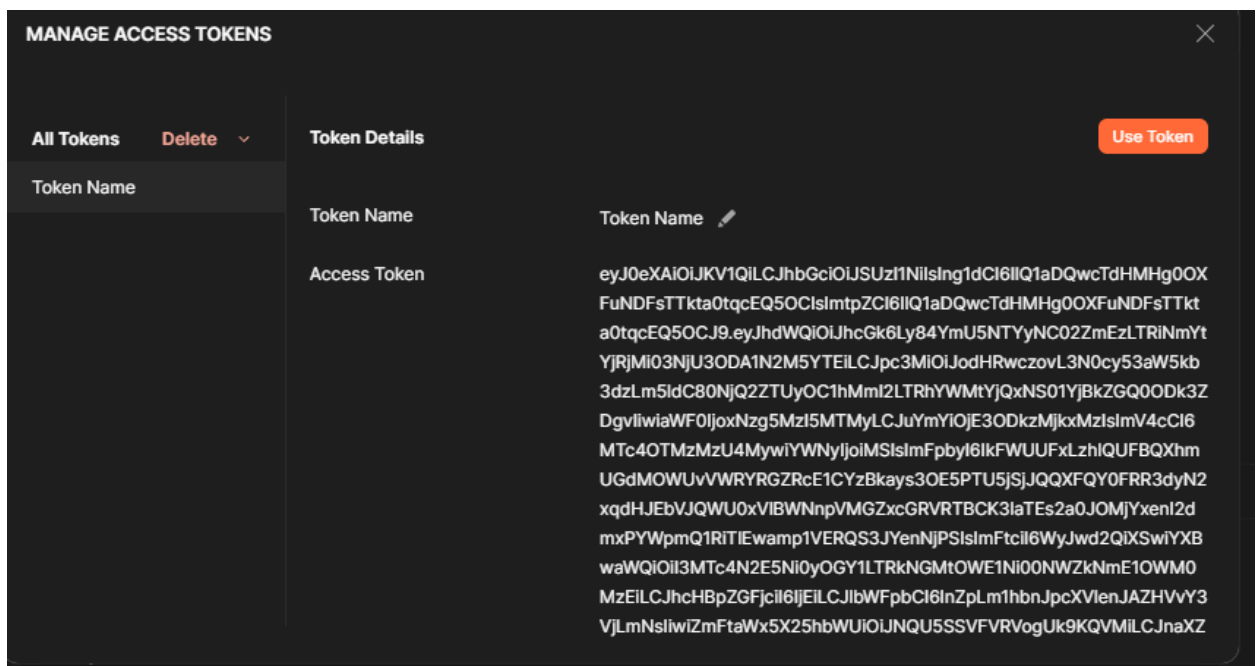
Ventana de login de Microsoft que Postman abre para autenticarse.



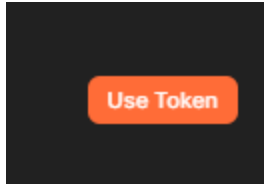
"Authentication complete": el login funcionó.



Administrar tokens: aparece el access token (JWT) generado, con el botón "Use Token".



Botón "Use Token": se aplica el token a la petición.



Respuestas de Postman: Código 200 el cual confirma la conexión

	200	729 ms	
	200	895.4 ms	
4c-9a56-45fd6a59c431&scope=api%3A%2F%2F8b302	302		
	200	542 ms	
	200	409.12 ms	
Globals Vault Tools			

Integrar los EC2

1. Entrar a ec2 y lanzar una instancia

Panel de EC2: muestra 0 instancias en ejecución; se pulsa "Lanzar la instancia"

EC2

Panel

Vista global de AWS ↗

Eventos

▼ **Instancias**

Instancias

Tipos de instancia

Plantillas de lanzamiento

Solicitudes de spot

Savings Plans

Instancias reservadas

Alojamientos dedicados

Reservas de capacidad

Gestor de capacidad **Novedad**

▼ **Imágenes**

AMI

Recursos

Está utilizando los siguientes recursos de Amazon EC2 en la región Estados Unidos

Instancias (en ejecución)	0	Alojamientos dedicados	0
Equilibradores de carga	0	Grupos de escalamiento	0
Grupos de ubicación	0	Instancias	0
Pares de claves	1	Reservas de capacidad	0

Lanzar la instancia

Para comenzar, lance una instancia de Amazon EC2, que es un servidor virtual en la nube.

Lanzar la instancia ▼ **Migrar un servidor** ↗

Nota: Sus instancias se lanzarán en la región Estados Unidos (Norte de Virginia)

2. Establecemos los parametros

Imagen de la instancia: nombre "Pedidos360Ec2", Amazon Linux 2023, 64 bits, apta para capa gratuita.

Amazon Linux macOS Ubuntu Windows Red Hat SUSE Linux Debian

Buscar más AMI

Incluidas las AMI de AWS, Marketplace y la comunidad

Imagen de máquina de Amazon (AMI)

AMI de Amazon Linux 2023 kernel-6.18
ami-0354c98ae10b02961 (64 bits (x86), uefi-preferred) / ami-0a157bd98d97a9589 (64 bits (Arm), uefi)
Virtualización: hvm Habilitado para ENA: true Tipo de dispositivo raíz: ebs

Apto para la capa gratuita ▼

Descripción

Amazon Linux 2023 (kernel-6.18) es un sistema operativo moderno y de uso general basado en Linux que contará con soporte hasta junio de 2029. Está optimizado para AWS y diseñado para proporcionar un entorno de ejecución seguro, estable y de alto desempeño para desarrollar y ejecutar sus aplicaciones en la nube.

Amazon Linux 2023 AMI 2023.12.20260909.0 x86_64 HVM kernel-6.18

Arquitectura	Modo de arranque	ID de AMI	Creation Date	Nombre de usuario	Proveedor verificado
64 bits (x86) ▼	uefi-preferred	ami-0354c98ae10b02961	2026-09-09	ec2-user	Proveedor verificado

3. Importante seleccionar el rango de IPs de nuestro proyecto, en este caso del 8081 al 8086

Grupo de seguridad Pedidos360-sg: SSH (22) solo desde mi IP y TCP 8081-8086 abierto a 0.0.0.0/0. AWS advierte sobre esa apertura.

Nombre del grupo de seguridad - *obligatorio*

Pedidos360-sg

Este grupo de seguridad se agregará a todas las interfaces de red. El nombre no se puede editar después de crear el grupo de seguridad. La longitud máxima es de 255 caracteres. Caracteres válidos: a-z, A-Z, 0-9, espacios y _-./()#,@[]+=&;{}!\$*

Descripción - *obligatorio* | [Información](#)

launch-wizard-1 created 2026-09-14T18:43:06.387Z

Reglas de grupos de seguridad de entrada

▼ Regla del grupo de seguridad 1 (TCP, 22, 181.43.151.60/32) Eliminar

Tipo Información	Protocolo Información	Rango de puertos Información
ssh	TCP	22
Tipo de origen Información	Nombre Información	Descripción - opcional Información
Mi IP	Agregue CIDR, lista de prefijos o grupo de seg. 181.43.151.60/32 ✕	por ejemplo, SSH para Admin Desktop

▼ Regla del grupo de seguridad 2 (TCP, 8081-8086, 0.0.0.0/0) Eliminar

Tipo Información	Protocolo Información	Rango de puertos Información
TCP personalizado	TCP	8081-8086
Tipo de origen Información	Origen Información	Descripción - opcional Información
Cualquier lugar	Agregue CIDR, lista de prefijos o grupo de seg. 	por ejemplo, SSH para Admin Desktop

4. Lanzamos instancia y anotamos la ip publica

Detalle de la instancia: en ejecución, tipo `t3.small` (posteriormente se modifico a una instancia `t3.medium` por problemas de conexion de la ec2).

i-08c71a59e38588e8b

ID de la instancia i-08c71a59e38588e8b	Dirección IPv4 pública 44.222.121.147 dirección abierta	Direcciones IPv4 privadas 172.31.71.71
Dirección IPv6 -	Estado de la instancia En ejecución	DNS público ec2-44-222-121-147.compute-1.amazonaws.com dirección abierta
Tipo de nombre de anfitrión Nombre de IP: ip-172-31-71-71.ec2.internal	Nombre DNS de IP privada (solo IPv4) ip-172-31-71-71.ec2.internal	Direcciones IP elásticas -
Responder al nombre DNS de recurso privado IPv4 (A)	Tipo de instancia t3.small	Hallazgo de AWS Compute Optimizer Suscribirse a AWS Compute Optimizer para recibir recomendaciones. Más información
Dirección IP asignada automáticamente 44.222.121.147 [IP pública]	ID de VPC vpc-0e45ec684b265b1f4	

5. Nos conectamos al EC2 a traves de ssh, usaremos powershell en la carpeta donde esta el par de llaves

Conexión SSH desde PowerShell: `ssh -i Pedidos360key.pem ec2-user@<ip>`, aceptando la huella del host.

Subir los microservicios al EC2

1. Nos dirigiremos a la carpeta de los microservicios en powershell y los compilamos en conjunto

Compilación: `.\mvnw.cmd clean package -DskipTests` en el repo del backend.

```
PS C:\Users\vicho\Downloads\aaaa\Encargo-Colegio-B0-Backend-main> .\mvnw.cmd clean package -DskipTests
[INFO] Scanning for projects...
[INFO] -----
[INFO] Reactor Build Order:
[INFO]
```

Resultado del build: los 6 servicios más apigateway

```
[INFO] apigateway 0.0.1-SNAPSHOT ..... SUCCESS [ 6.772 s]
[INFO] colegipo 0.0.1-SNAPSHOT ..... SUCCESS [ 4.916 s]
[INFO] Servicio Asistencia 1.0.0 ..... SUCCESS [ 3.700 s]
[INFO] servicio-matricula 0.0.1-SNAPSHOT ..... SUCCESS [ 6.001 s]
[INFO] Servicio Academico 1.0.0 ..... SUCCESS [ 4.056 s]
[INFO] Servicio Reportes 1.0.0 ..... SUCCESS [ 4.543 s]
[INFO] Servicio Mensajería 1.0.0 ..... SUCCESS [ 4.338 s]
[INFO] Colegio System - Parent POM 1.0.0 ..... SUCCESS [ 0.064 s]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 35.175 s
[INFO] Finished at: 2026-09-14T16:03:47-03:00
[INFO] -----
PS C:\Users\vicho\Downloads\aaaa\Encargo-Colegio-B0-Backend-main> |
```

2. Preparamos la conexión de ssh hacia el ec2

Variables de PowerShell: `$REPO`, `$KEY` y `$EC2`, para no repetir rutas en cada comando.

```
PS C:\Users\vicho\Downloads\aaaa\Encargo-Colegio-B0-Backend-main> $REPO = "C:\Users\vicho\Downloads\aaaa\Encargo-Colegio-B0-Backend-main"
PS C:\Users\vicho\Downloads\aaaa\Encargo-Colegio-B0-Backend-main> $KEY = "C:\Users\vicho\Downloads\Pedidos360key.pem"
PS C:\Users\vicho\Downloads\aaaa\Encargo-Colegio-B0-Backend-main> $EC2 = "ec2-user@44.222.121.147"
```

3. Subimos cada microservicio a su respectiva carpeta del ec2

Primer scp: sube el JAR de usuarios a `~/usuarios/`.

```
PS C:\Users\vicho\Downloads\aaaa\Encargo-Colegio-B0-Backend-main> scp -i $KEY "$REPO\servicio-usuarios\target\colegipo-0.0.1-SNAPSHOT.jar" "$EC2:~/usuarios/"
colegipo-0.0.1-SNAPSHOT.jar                               100% 67MB 11.1MB/s 00:06
PS C:\Users\vicho\Downloads\aaaa\Encargo-Colegio-B0-Backend-main> |
```

repetimos para cada microservicio

scp de los demás servicios: matrícula, académico, reportes y mensajería, cada uno a su carpeta.

```

PS C:\Users\vicho\Downloads\aaaa\Encargo-Colegio-B0-Backend-main> scp -i $KEY "$REPO\servicio-matricula\target\servicio-
matricula-0.0.1-SNAPSHOT.jar" "${EC2}:~/matricula/"
servicio-matricula-0.0.1-SNAPSHOT.jar                                100% 71MB 11.8MB/s 00:06
PS C:\Users\vicho\Downloads\aaaa\Encargo-Colegio-B0-Backend-main> scp -i $KEY "$REPO\servicio-academico\target\servicio-
academico-1.0.0.jar" "${EC2}:~/academico/"
servicio-academico-1.0.0.jar                                       100% 55MB 11.0MB/s 00:04
PS C:\Users\vicho\Downloads\aaaa\Encargo-Colegio-B0-Backend-main> scp -i $KEY "$REPO\servicio-reportes\target\servicio-r
eportes-1.0.0.jar" "${EC2}:~/reportes/"
servicio-reportes-1.0.0.jar                                       100% 62MB 11.6MB/s 00:05
PS C:\Users\vicho\Downloads\aaaa\Encargo-Colegio-B0-Backend-main> scp -i $KEY "$REPO\servicio-mensajeria\target\servicio-
mensajeria-1.0.0.jar" "${EC2}:~/mensajeria/"
servicio-mensajeria-1.0.0.jar                                     100% 72MB 11.8MB/s 00:06
PS C:\Users\vicho\Downloads\aaaa\Encargo-Colegio-B0-Backend-main> |

```

- Verificamos que si se encuentran en las carpetas, usaremos la sesion ssh que usamos antes

ls -lh en EC2: confirma que cada carpeta contiene su JAR.

```

[ec2-user@ip-172-31-71-71 ~]$ ls -lh usuarios/ asistencia/ matricula/ academico/ reportes/ mensajeria/
academico/:
total 55M
-rw-rw-r--. 1 ec2-user ec2-user 55M Sep 14 19:19 servicio-academico-1.0.0.jar

asistencia/:
total 65M
-rw-rw-r--. 1 ec2-user ec2-user 65M Sep 14 19:23 servicio-asistencia-1.0.0.jar

matricula/:
total 72M
-rw-rw-r--. 1 ec2-user ec2-user 72M Sep 14 19:18 servicio-matricula-0.0.1-SNAPSHOT.jar

mensajeria/:
total 73M
-rw-rw-r--. 1 ec2-user ec2-user 73M Sep 14 19:19 servicio-mensajeria-1.0.0.jar

reportes/:
total 63M
-rw-rw-r--. 1 ec2-user ec2-user 63M Sep 14 19:23 servicio-reportes-1.0.0.jar

usuarios/:
total 68M
-rw-rw-r--. 1 ec2-user ec2-user 68M Sep 14 19:17 colegipo-0.0.1-SNAPSHOT.jar
[ec2-user@ip-172-31-71-71 ~]$ |

```

- Y ahora creamos los archivos systemd

Archivo systemd: unidad `pedidos-usuarios.service` con `DB_PASSWORD`, `SERVER_PORT=8081`, `ExecStart` con `java -jar` y `Restart=always`.

```
[Unit]
Description=Pedidos360 Usuarios
After=network.target

[Service]
Environment="DB_PASSWORD=12345678"
Environment="SERVER_PORT=8081"
ExecStart=/usr/bin/java -jar /home/ec2-user/usuarios/colegipo-0.0.1-SNAPSHOT.jar
Restart=always
User=ec2-user

[Install]
WantedBy=multi-user.target
SERVICE
[ec2-user@ip-172-31-71-71 ~]$ cat /etc/systemd/system/pedidos-usuarios.service
[Unit]
Description=Pedidos360 Usuarios
After=network.target

[Service]
Environment="DB_PASSWORD=12345678"
Environment="SERVER_PORT=8081"
ExecStart=/usr/bin/java -jar /home/ec2-user/usuarios/colegipo-0.0.1-SNAPSHOT.jar
Restart=always
User=ec2-user

[Install]
WantedBy=multi-user.target
[ec2-user@ip-172-31-71-71 ~]$ |
```

repetimos con cada microservicio

6. Recargamos systemd para que reconozca los nuevos servicios y los activamos

Activación: **daemon-reload** y **enable** de los 6 servicios, con sus symlinks creados.

```
[ec2-user@ip-172-31-71-71 ~]$ sudo systemctl daemon-reload
[ec2-user@ip-172-31-71-71 ~]$ sudo systemctl enable pedidos-usuarios pedidos-asistencia pedidos-matricula pedidos-academico pedidos-reportes pedidos-mensajeria
Created symlink /etc/systemd/system/multi-user.target.wants/pedidos-usuarios.service → /etc/systemd/system/pedidos-usuarios.service.
Created symlink /etc/systemd/system/multi-user.target.wants/pedidos-asistencia.service → /etc/systemd/system/pedidos-asistencia.service.
Created symlink /etc/systemd/system/multi-user.target.wants/pedidos-matricula.service → /etc/systemd/system/pedidos-matricula.service.
Created symlink /etc/systemd/system/multi-user.target.wants/pedidos-academico.service → /etc/systemd/system/pedidos-academico.service.
Created symlink /etc/systemd/system/multi-user.target.wants/pedidos-reportes.service → /etc/systemd/system/pedidos-reportes.service.
Created symlink /etc/systemd/system/multi-user.target.wants/pedidos-mensajeria.service → /etc/systemd/system/pedidos-mensajeria.service.
[ec2-user@ip-172-31-71-71 ~]$ |
```

7. Los arrancamos

systemctl status 'pedidos-*': el servicio **pedidos-academico** aparece *enabled* y *active (running)* desde hace 32 s. Corre con **java -jar** y sus logs de arranque se ven abajo.

```

[ec2-user@ip-172-31-71-71 ~]$ sudo systemctl status 'pedidos-*' --no-pager
● pedidos-academico.service - Pedidos360 Academico
   Loaded: loaded (/etc/systemd/system/pedidos-academico.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-09-14 19:34:02 UTC; 32s ago
     Main PID: 27886 (java)
        Tasks: 23 (limit: 2255)
       Memory: 150.1M
          CPU: 11.444s
      CGroup: /system.slice/pedidos-academico.service
              └─27886 /usr/bin/java -jar /home/ec2-user/academico/servicio-academico-1.0.0.jar

Sep 14 19:34:25 ip-172-31-71-71.ec2.internal java[27886]: 2026-09-14T19:34:25.854Z INFO 27886 --- [servi
Sep 14 19:34:25 ip-172-31-71-71.ec2.internal java[27886]: 2026-09-14T19:34:25.885Z INFO 27886 --- [servi
Sep 14 19:34:25 ip-172-31-71-71.ec2.internal java[27886]: 2026-09-14T19:34:25.885Z INFO 27886 --- [servi

```

8. Abrimos una nueva terminal de powershell y verificamos la conexión

curl a :8081/api/usuarios: responde "(401) No autorizado". Eso confirma que el microservicio de usuarios rechaza peticiones sin token, es decir, Entra ID está activo.

```

PS C:\WINDOWS\System32> curl http://44.222.121.147:8081/api/usuarios
curl : Error en el servidor remoto: (401) No autorizado.
En línea: 1 Carácter: 1
+ curl http://44.222.121.147:8081/api/usuarios
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebExc
  eption
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand

PS C:\WINDOWS\System32> |

```

este mensaje de error es el que buscamos, ya que el error 401 significa que no contamos con la autorización para conectarnos, lo cual significa que el Entra ID está activo, ahora hacemos un **curl a los otros microservicios**.
los tres devuelven el mismo 401.

```

PS C:\WINDOWS\System32> curl http://3.236.96.168:8082/api/asistencia
curl : Error en el servidor remoto: (401) No autorizado.
En línea: 1 Carácter: 1
+ curl http://3.236.96.168:8082/api/asistencia
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebExc
  eption
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand

PS C:\WINDOWS\System32> curl http://3.236.96.168:8083/api/matricula
curl : Error en el servidor remoto: (401) No autorizado.
En línea: 1 Carácter: 1
+ curl http://3.236.96.168:8083/api/matricula
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebExc
  eption
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand

PS C:\WINDOWS\System32> curl http://3.236.96.168:8084/api/academico
curl : Error en el servidor remoto: (401) No autorizado.
En línea: 1 Carácter: 1
+ curl http://3.236.96.168:8084/api/academico
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebExc
  eption
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand

```

API GATEWAY

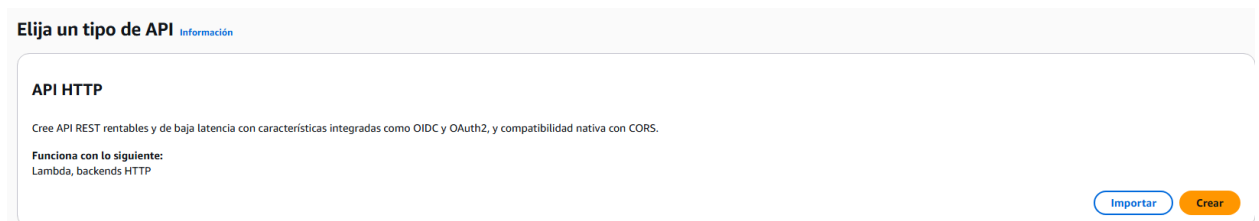
1. Entramos a api gateway en aws console

Consola de API Gateway: pantalla de inicio con el botón "Crear una API".



2. Creamos API http

Tipo de API: se elige "API HTTP" y se pulsa "Crear".



3. En integraciones http ponemos la IP publica del ec2 junto con los puertos de los servicios

Integraciones HTTP: se agregan las integraciones con método ANY, apuntando a la IP pública del EC2 en los puertos 8081, 8082, 8083 y siguientes.



4. Configuramos las rutas

Configuración de rutas: seis rutas `/api/<servicio>/{proxy+}` (usuarios, asistencia, matricula, academico, reportes, mensajes), cada una hacia su puerto del 8081 al 8086.

Configurar rutas - opcional

Configurar rutas [Información](#)

API Gateway usa rutas para exponer las integraciones a los clientes de su API. Para las API HTTP, las rutas constan de dos partes: un método HTTP (p. ej., GET /pets) y una ruta de recurso. Puede definir métodos HTTP específicos para su integración (GET, POST, PUT, PATCH, HEAD, OPTIONS y DELETE) o utilizar el método ANY para que coincida con todos los métodos que no haya definido en un recurso determinado.

Método	Ruta de recurso	Destino de integración	
ANY	/api/usuarios/{proxy+}	ANY http://IP-3.238.133.251-EC2:8081	Eliminar
ANY	/api/asistencia/{proxy+}	ANY http://IP-3.238.133.251-EC2:8082	Eliminar
ANY	/api/matricula/{proxy+}	ANY http://IP-3.238.133.251-EC2:8083	Eliminar
ANY	/api/academico/{proxy+}	ANY http://IP-3.238.133.251-EC2:8084	Eliminar
ANY	/api/reportes/{proxy+}	ANY http://IP-3.238.133.251-EC2:8085	Eliminar
ANY	/api/mensajes/{proxy+}	ANY http://IP-3.238.133.251-EC2:8086	Eliminar

[Agregar ruta](#)

[Cancelar](#) [Revisar y crear](#) [Anterior](#) [Siguiente](#)

5. La creamos

API creada: mensaje verde "Successfully created API pedidos360-api" y el árbol de rutas bajo `/api`, con el botón "Implementación".

Successfully created API pedidos360-api (yesinmq43).

Rutas [Etapas: -](#) [Implementación](#)

Rutas para pedidos360-api [Crear](#)

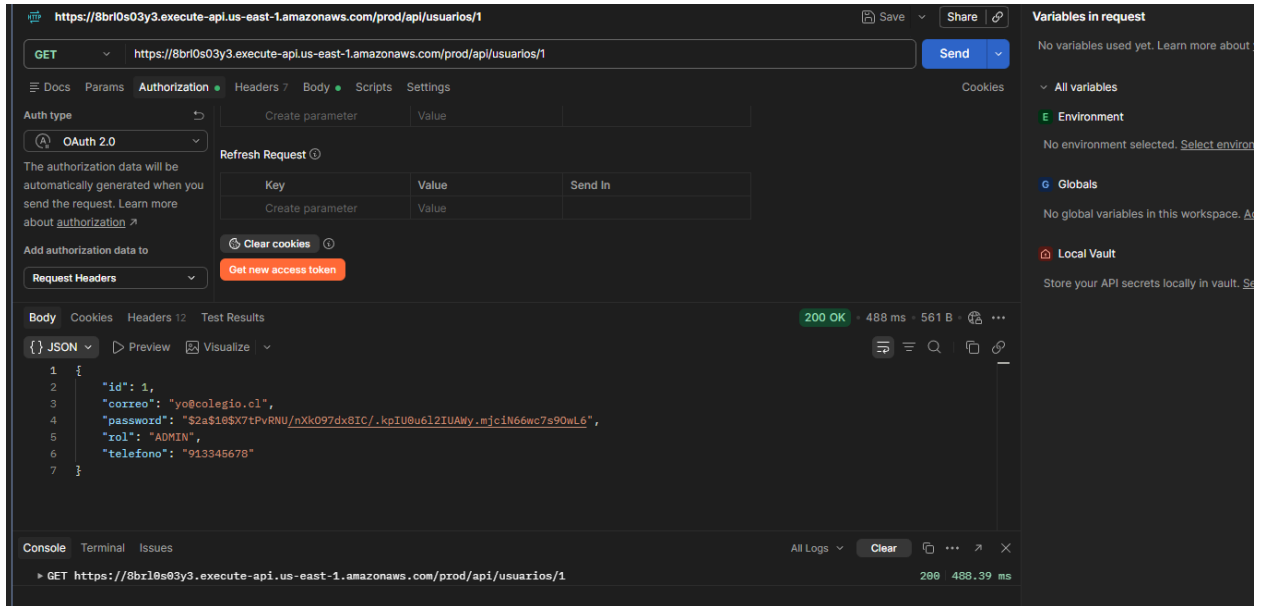
Buscar

- ▼ /api
 - ▼ /reportes
 - ▼ /{proxy+}ANY
 - ▼ /academico
 - ▼ /{proxy+}ANY
 - ▼ /matricula
 - ▼ /{proxy+}ANY

Elija una ruta.

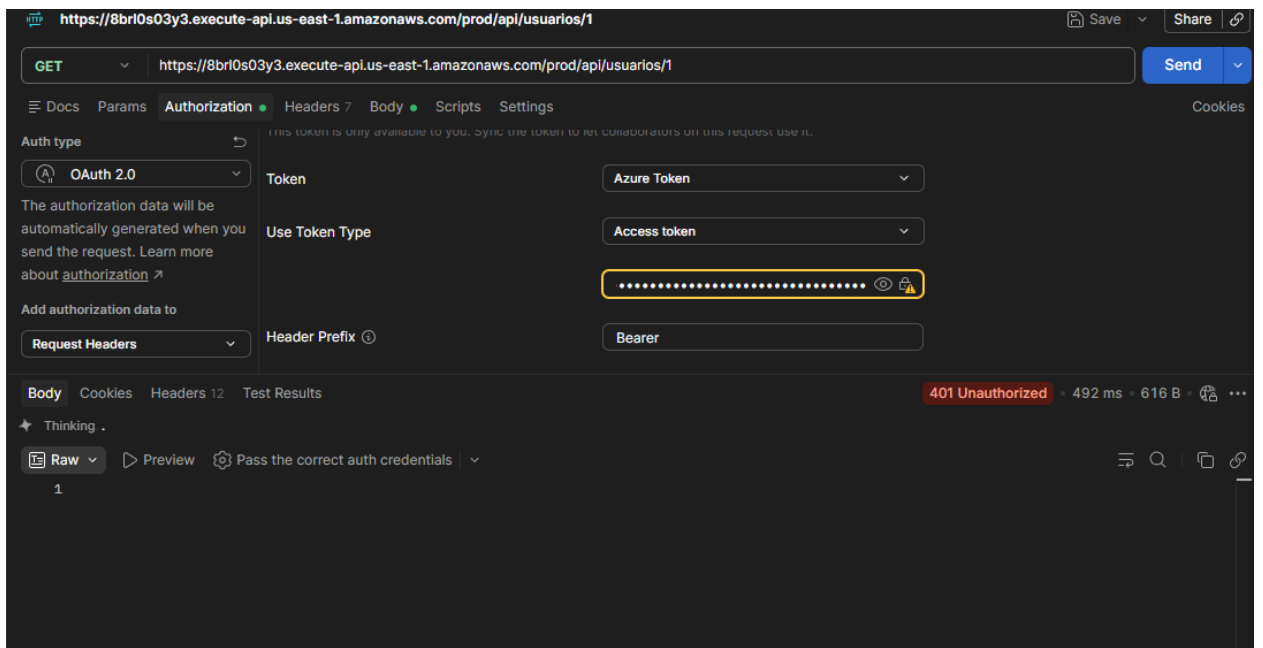
6. Probamos con postman

Postman con token: un GET a `.../prod/api/usuarios/1` con OAuth 2.0 devuelve **200 OK** y un JSON con id, correo, password (hash), rol ADMIN y teléfono.



aquí una vez ponemos el bearer token nos devolvera la informacion que se encuentra en los servicios del EC2 a traves de una peticion al Gateway

Postman sin token válido: la misma petición devuelve **401 Unauthorized**, con el mensaje "Pass the correct auth credentials". En la captura el campo de token aparece con "Azure Token" seleccionado, así que el texto "sin token" no se ve reflejado exactamente. Convendría aclararlo en el informe.



sin token, tira error 401

JWT AUTHORIZER

1. Nos dirigimos a autorizadores del gateway y agregamos uno
Pestaña Autorización: la ruta `ANY /api/asistencia/{proxy+}` aún no tiene autorizador. Desde ahí se puede "Crear y asociar un autorizador".

Se actualizó correctamente la integración de "2js7153".

Autorización Etapas: - Implementación

[Asociar autorizadores a rutas](#) [Administrar autorizadores](#)

Rutas para api360

Buscar

- ▼ /api
 - ▼ /asistencia
 - ▼ /(proxy+)
 - ANY**
 - ▼ /reportes
 - ▼ /(proxy+)
 - ANY
 - ▼ /academico
 - ▼ /(proxy+)
 - ANY

No hay ningún autorizador asociado a ANY /api/asistencia/{proxy+}

No hay ningún autorizador asociado a esta ruta. Puede asociar un autorizador existente o crear y asociar uno nuevo.

[Crear y asociar un autorizador](#)

- 0 -

Seleccionar un autorizador existente

Asociar autorizador

Seleccione un autorizador para ver sus detalles.

2. En url de emisor agregamos el id del tenant que usamos
Configuración del autorizador: tipo JWT, nombre `jwt`, origen `$request.header.Authorization` y URL del emisor `https://login.microsoftonline.com/<tenant>/v2.0`. El campo Audiencia aún está vacío.

Tipo de autorizador [Información](#)

☒ **JWT**
Utilice los tokens web JSON (JWT) para controlar el acceso a la API.

☐ **Lambda**
Utilice una función Lambda para controlar el acceso a la API.

If you want to attach IAM authorization, go back to [Attach authorizers to routes](#) and select IAM (built-in) from the list of existing authorizers. We provide IAM authorization for you through a built-in authorizer in the c

Configuración del autorizador

Nombre
Escriba un nombre para el autorizador.

jwt

Origen de identidad
Escriba una expresión de selección que defina el origen del token.

\$request.header.Authorization

URL del emisor
Escriba la URL del emisor de su proveedor de identidad, que suele encontrarse en el campo emisor del punto de enlace de metadatos conocido del servidor de autorización.

https://login.microsoftonline.com/4646e528-a2b6-4aac-b415-5b0ddd4897d8/v2.0

Audiencia
Escriba los ID de clientes registrados con proveedores de identidad o cualquier cadena arbitraria en la reclamación de público de JWT que el autorizador debe verificar.
No hay público asociado con el autorizador

[Agregar público](#)

3. En audiencia agregamos el app id de pedidos-api (el de azure)

Audiencia: se agrega el App ID de Pedidos360-API (8be95624-...).

Audiencia
Escriba los ID de clientes registrados con proveedores de identidad o cualquier cadena arbitraria en la reclamación de público de JWT que el autorizador debe verificar.

8be95624-6fa3-4b6f-b4c2-76578057c9a1

Eliminar

Agregar público

4. Asociamos el autorizador a las rutas

Autorizadores asociados: las rutas de mensajes, usuarios y matrícula muestran la etiqueta "JWT Auth". El detalle repite emisor y audiencia.

Asociar autorizadores a rutas | Administrar autorizadores

Rutas para api360

Buscar

ANY JWT Auth

▼ /mensajes

▼ /(proxy+)

ANY JWT Auth

▼ /usuarios

▼ /(proxy+)

ANY JWT Auth

▼ /matricula

▼ /(proxy+)

ANY JWT Auth

Autorizador para la ruta ANY /api/matricula/{proxy+}

Nombre de autorizador jwt **Tipo de autorizador** JWT

Origen de identidad
Cuando se invoca a este autorizador, API Gateway utilizará esta expresión de selección para determinar \$request.header.Authorization

Emisor
El URI del emisor del proveedor de identidad
https://login.microsoftonline.com/4646e528-a2b6-4aac-b415-5b0ddd4897d8/v2.0

Audiencia
El público asociado con este autorizador

- 8be95624-6fa3-4b6f-b4c2-76578057c9a1

Ámbitos de autorización

5. Y en cors agregamos al localhost como origen, con crud y headers de autorizacion

Configuración de CORS:

- Origen <http://localhost:5173>.
- Métodos GET, POST, OPTIONS, PUT y DELETE.
- Headers [authorization](#) y [content-type](#).
- Credentials en NO.

Uso compartido de recursos entre orígenes

Configurar CORS [Información](#)
CORS allows resources from different domains to be loaded by browsers. If you configure CORS for an API, API Gateway ignores CORS headers returned from your backend integration. See our [CORS documentation](#) for more details.

Access-Control-Allow-Origin

Agregar

Access-Control-Allow-Methods

Elija los métodos permitidos

GET POST OPTIONS PUT DELETE

Access-Control-Max-Age

Access-Control-Allow-Headers

Escribir un valor para los encabezados permitidos

authorization content-type

Access-Control-Expose-Headers

Escribir un valor para los encabezados expuestos

Agregar

Access-Control-Allow-Credentials
☐ NO

Cancelar Guardar

Frontend MSAL

1. Nos dirigimos a la carpeta del frontend e instalamos el autenticador web de azure

Instalación: `npm install @azure/msal-browser @azure/msal-react react-router-dom --legacy-peer-deps` en la carpeta `front`, con 0 vulnerabilidades.

```
C:\Users\vicho\Downloads\aaaa\front>npm install @azure/msal-browser @azure/msal-react react-router-dom --legacy-peer-deps
added 3 packages, removed 1 package, changed 2 packages, and audited 82 packages in 4s
14 packages are looking for funding
  run 'npm fund' for details
found 0 vulnerabilities
```

2. Creamos el archivo env con los datos de azure que guardamos

Archivo .env: define `VITE_AZURE_CLIENT_ID` (el del frontend), `VITE_AZURE_TENANT_ID`, `VITE_API_BASE_URL` (la URL de API Gateway) y `VITE_API_SCOPE` (`api://.../access_as_user`).

```
front > .env
1 VITE_AZURE_CLIENT_ID=71787a96-28f5-4d4c-9a56-45fd6a59c431
2 VITE_AZURE_TENANT_ID=4646e528-a2b6-4aac-b415-5b0ddd4897d8
3 VITE_API_BASE_URL=https://8br10s03y3.execute-api.us-east-1.amazonaws.com
4 VITE_API_SCOPE=api://8be95624-6fa3-4b6f-b4c2-76578057c9a1/access_as_user
5
```

3. Creamos un archivo authconfig para que el programa se comuniqué con azure

authConfig.js: exporta **msalConfig** (clientId, authority con el tenant, redirectUri, caché en sessionStorage), **loginRequest** (scopes **openid** y **profile**) y **protectedResources** (endpoint y scope de la API).

```
ht > src > JS authConfig.js > msalConfig > cache > cacheLocation
1 // src/authConfig.js
2 // Configuración de Microsoft Authentication Library (MSAL) para Azure AD
3
4 export const msalConfig = {
5   auth: {
6     clientId: import.meta.env.VITE_AZURE_CLIENT_ID,
7     authority: `https://login.microsoftonline.com/${import.meta.env.VITE_AZURE_TENANT_ID}`,
8     redirectUri: window.location.origin,
9     postLogoutRedirectUri: window.location.origin,
10   },
11   cache: {
12     cacheLocation: 'sessionStorage',
13     storeAuthStateInCookie: false,
14   },
15 };
16
17 export const loginRequest = {
18   scopes: ['openid', 'profile'],
19 };
20
21 export const protectedResources = {
22   api: {
23     endpoint: import.meta.env.VITE_API_BASE_URL,
24     scopes: [import.meta.env.VITE_API_SCOPE],
25   },
26 };
27
```

4. Creamos un public client application para incorporar msal en la app

main.jsx: se crea **new PublicClientApplication(msalConfig)** y se envuelve **<App />** en **<MsalProvider>**.

```
const msalInstance = new PublicClientApplication(msalConfig);

<StrictMode>
  <MsalProvider instance={msalInstance}>
    <App />
  </MsalProvider>
</StrictMode>,
);
```

5. Redirigimos el login a azure

Componente AuthButtons: usa `useMsal()` y `handleLogin` llama a `instance.loginRedirect(loginRequest)`, que redirige el login a Azure.

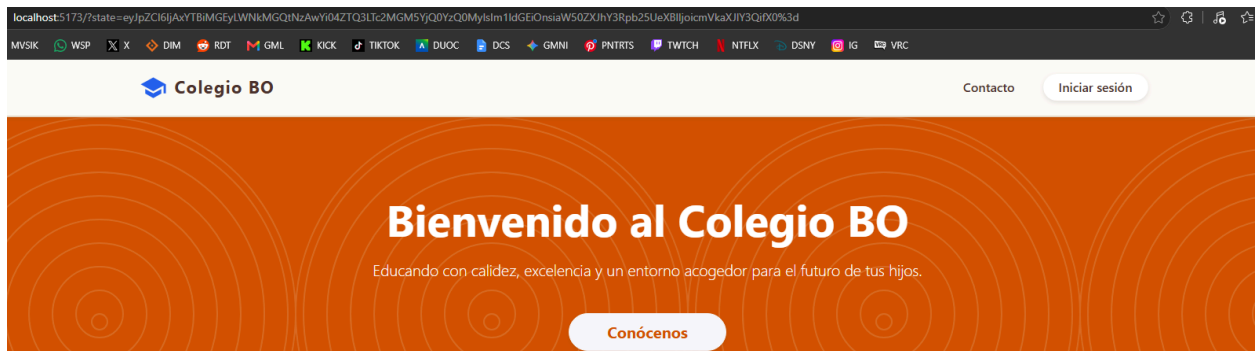
```
import { useMsal } from '@azure/msal-react';
import { loginRequest } from './authConfig';

export default function AuthButtons() {
  const { instance, accounts } = useMsal();
  const isAuthenticated = accounts.length > 0;

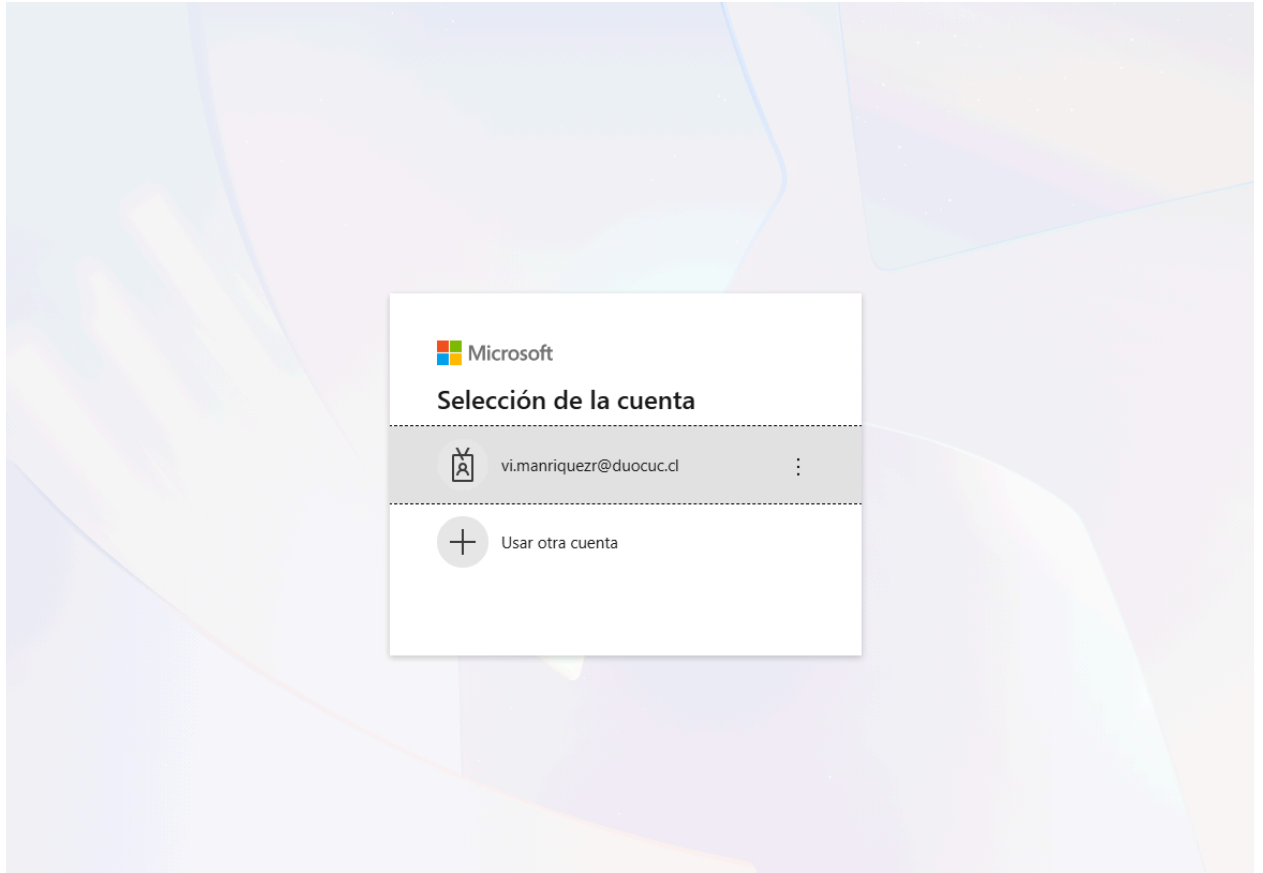
  const handleLogin = () => {
    instance.loginRedirect(loginRequest).catch(console.error);
  };
}
```

6. Aquí ya nos pide el inicio de sesión a través de Azure

Página de inicio del "Colegio BO" en `localhost:5173`: se ve el botón "Iniciar sesión". La URL ya trae el parámetro `state` de MSAL, es decir, el flujo de login se inició.



Pantalla de Microsoft: "Selección de la cuenta" con la cuenta institucional de Duoc UC.



7. Y aquí nos dejó entrar con la cuenta de Microsoft que utilizamos

Resultado final: la app muestra "Bienvenido de nuevo VICENTE MANRIQUEZ ROJAS", el rol asignado "Usuario" y el aviso "API Gateway respondió correctamente (200 OK)". Todo el flujo funciona de punta a punta.

Bienvenido

Bienvenido de nuevo
VICENTE MANRIQUEZ ROJAS

Rol asignado: **Usuario**

✓ API Gateway respondió correctamente (200 OK)

[Cerrar Sesión](#)

Plataforma de Gestión Académica